



CS61C

Great Ideas
in
Computer Architecture
(a.k.a. Machine Structures)



Assistant
Teaching Professor
Lisa Yan

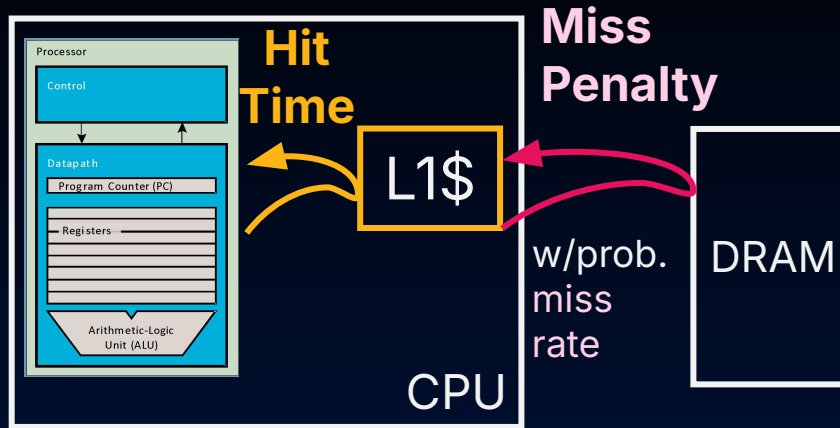
Caches II: Fully Associative



AMAT: Average Memory Access Time

- AMAT
- Cache Terminology
- Fully Associative Cache
- Warming up the Fully Associative Cache
- Line Replacement Policies
- [if time] Write Policies

Average Memory Access Time



Average Access Memory Time

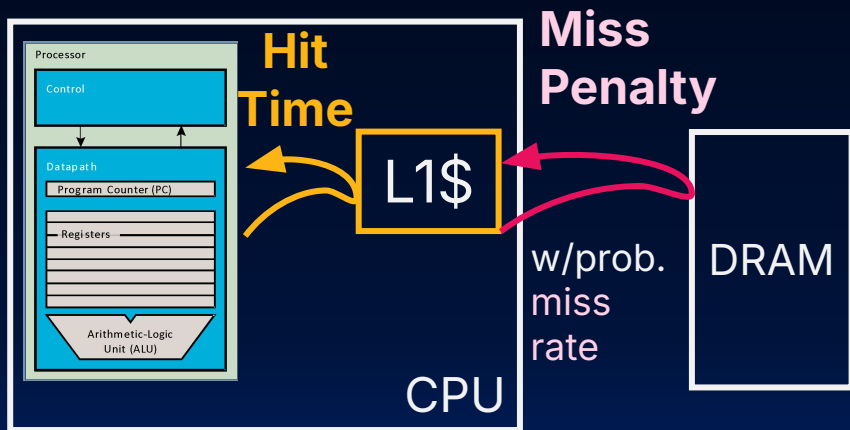
$$AMAT = \text{Hit Time} + \text{Miss Rate} \times \text{Miss Penalty}$$

Goal: Design caches to reduce average memory access time.

What is the Average Memory Access Time?

Assume just one L1 cache:

- L1 Hit Time = 1 cycle
- L1 Miss rate = 5%
- L1 Miss penalty = 200 cycles



What is the **Average Memory Access Time**, in # cycles?

- Hint 1: Think probability and expectation!
- Hint 2: On cache miss,
Total Time to retrieve data
= Hit time + Miss penalty.

- A. 1
- B. 1.95
- C. 2
- D. 21
- E. 10.95
- F. 11
- G. Something else

What is the Average Memory Access Time?

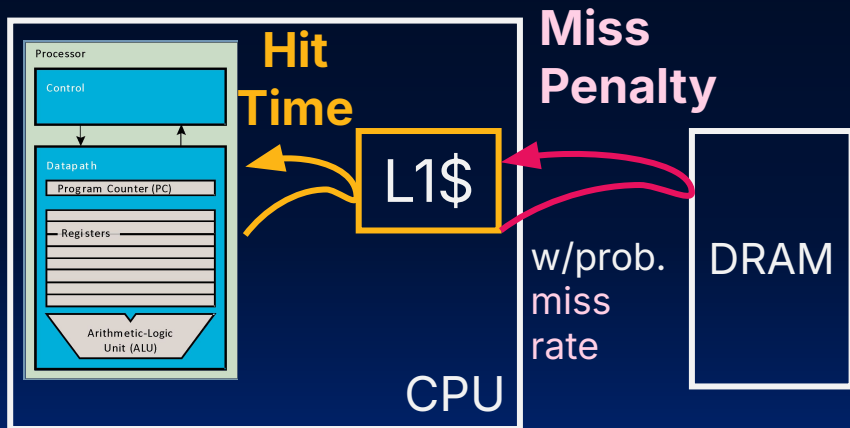
Without L2 cache, assume:

- L1 Hit Time = 1 cycle
- L1 Miss rate = 5%
- L1 Miss penalty = 200 cycles

What is the **Average Memory Access Time**, in # cycles?

- Hint 1: Think probability and expectation!
- Hint 2: On cache miss, Total Time to retrieve data = Hit time + Miss penalty.

$$\begin{aligned} AMAT &= \text{Hit Rate} \cdot (\text{Hit Time}) \\ &\quad + \text{Miss Rate} \cdot (\text{Hit Time} + \text{Miss Penalty}) \\ &= \text{Hit Time} + \text{Miss Rate} \cdot (\text{Miss Penalty}) \\ &= (1) + .05(200) = \mathbf{11} \end{aligned}$$



L15: What is the Average Memory Access Time, in cycles?

A. 1

0%

B. 1.95

0%

C. 2

0%

D. 21

0%

E. 10.95

0%

F. 11

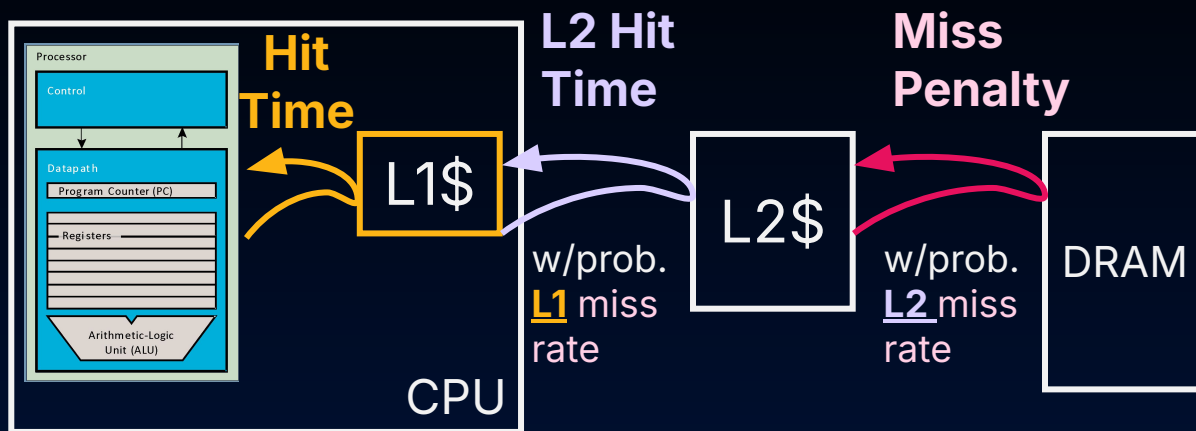
0%

G. Something else

0%



Average Memory Access Time: Recursive Formula



$$AMAT = (L1 \text{ Hit Time}) + L1 \text{ Miss Rate} \cdot (L1 \text{ Avg. Miss Penalty})$$

recursive



$$L2 \text{ Hit Time} + L2 \text{ Miss Rate} \cdot (L2 \text{ Miss Penalty})$$

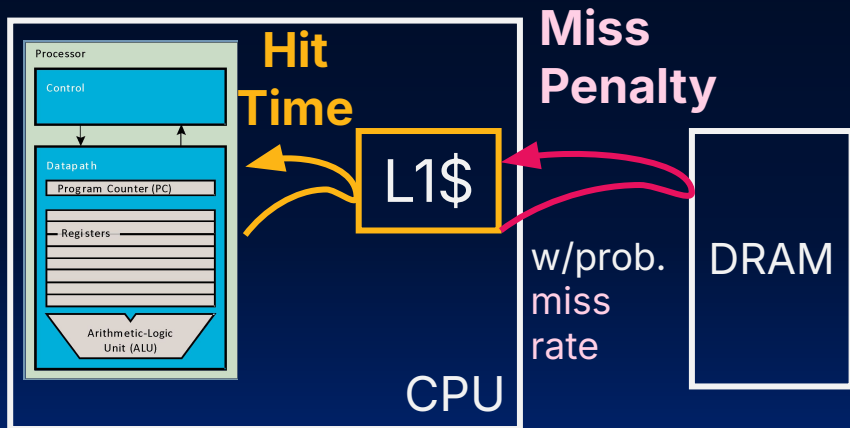
Example 1: Without L2 Cache (from before)

Without L2 cache, assume:

- L1 Hit Time = 1 cycle
- L1 Miss rate = 5%
- L1 Miss penalty = 200 cycles

AMAT

$$\begin{aligned} &= \text{L1 hit time} \\ &\quad + (\text{L1 Miss rate}) * (\text{L1 Miss Penalty}) \\ &= 1 + 0.05 \times 200 \\ &= \underline{11 \text{ cycles}} \end{aligned}$$



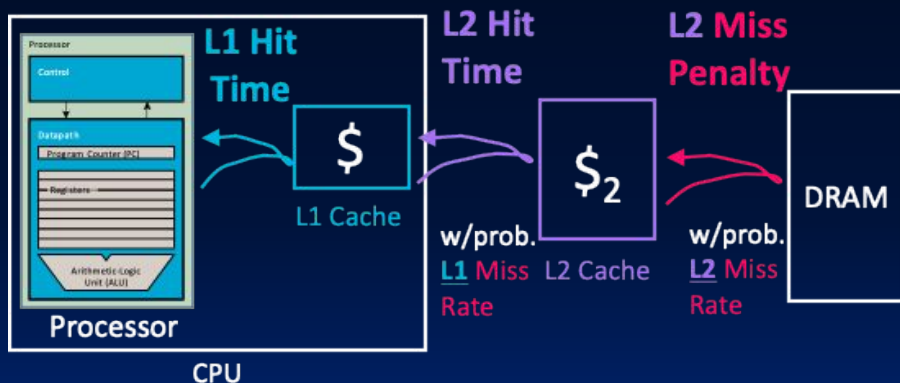
Example 2: With L2 Cache

With L2 cache, assume:

- L1 Hit Time = 1 cycle
- L1 Miss rate = 5%
- L2 Hit Time = 5 cycles
- L2 Miss rate = 15%
- L2 Miss penalty = 200 cycles

$$\text{L1 (average) miss penalty} = 5 + 0.15 * 200 = \underline{\underline{35}}$$

$$\begin{aligned} \text{AMAT} &= 1 + 0.05 * \underline{\underline{35}} \\ &= 2.75 \text{ cycles} \end{aligned}$$



4x faster with L2 cache!
(2.75 vs. 11)

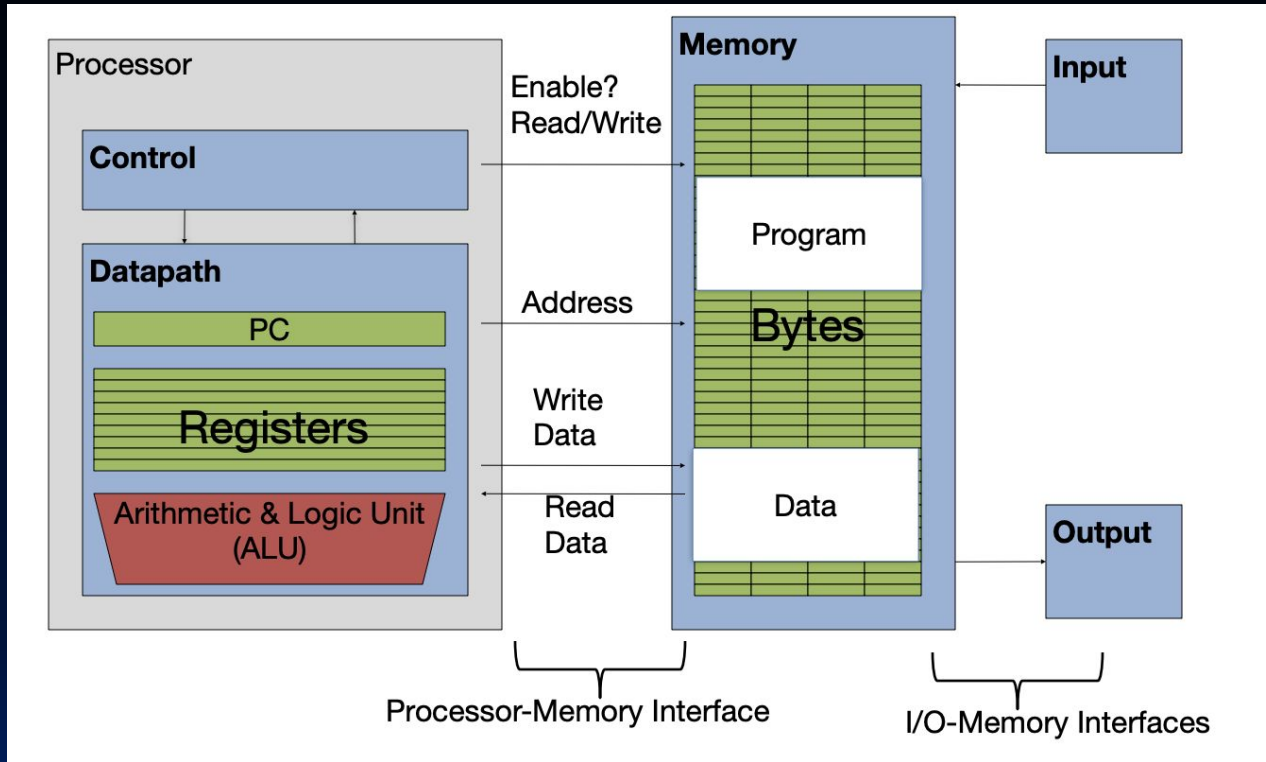
L2\$ tends to receive only the "hard" memory accesses (the ones that miss in the L1\$).



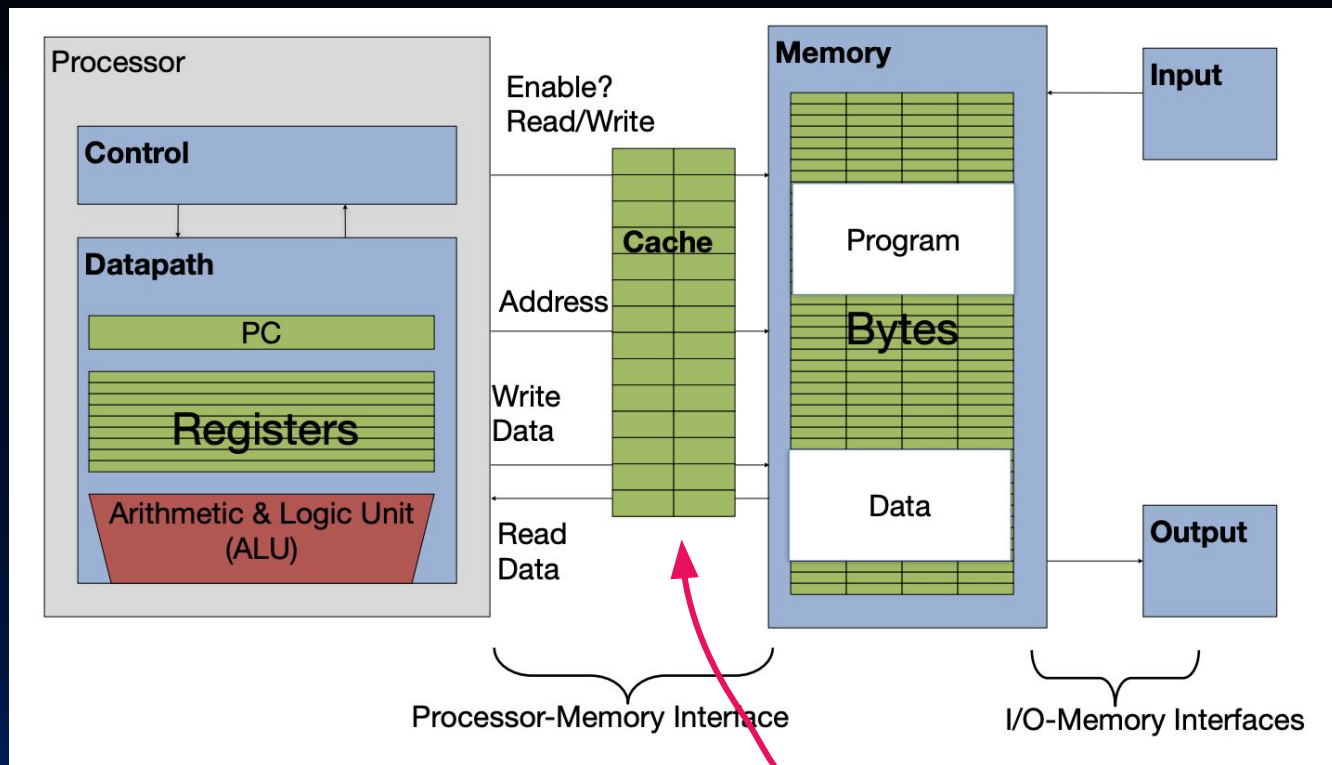
Cache Terminology

- AMAT
- Cache Terminology
- Fully Associative Cache
- Warming up the Fully Associative Cache
- Line Replacement Policies
- [if time] Write Policies

Computer without Cache



Computer with Cache



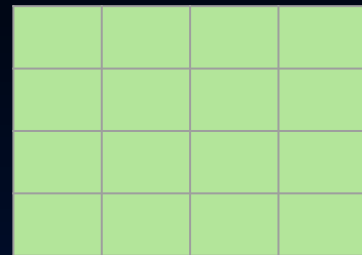
Lines (i.e., blocks) of data are copied from memory to the cache.



Cache Terminology

Spatial locality: When we bring data from memory into the cache, we do so at the granularity of a cache line (or cache block).

- Cache **line**/block:
 - A single entry in the cache
- **Line size** / block size:
 - # bytes per cache line
- "16B" cache
 - Total # data bytes that can be stored in a cache, excluding "metadata"



Apple M1: 64B cache lines in L1 (usual), 128B lines in L2 [[GoFetch](#)]



Memory Access without Cache

Load word instruction:

t0

1234

lw t0 0(t1)

t1

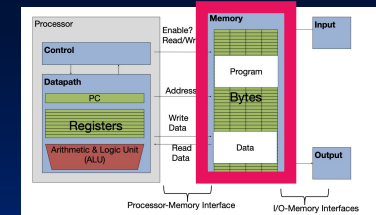
0x12F0

Memory[0x12F0] = 1234

Memory access **without cache**:

1. Processor issues address 0x12F0 to memory
2. **Memory** reads 1234 @ address 0x12F0

3. **Memory** sends 1234 to Processor
4. Processor loads 1234 into register t0





Memory Access with Cache

Load word instruction:

t0

1234

Memory[0x12F0] = 1234

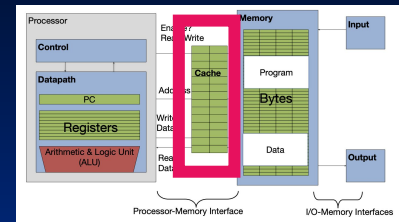
lw t0 0(t1)

t1

0x12F0

Memory access **with cache**:

1. Processor issues address 0x12F0 to **cache**
2. **Cache** checks for copy of data, addr. 0x12F0
 - a. If **cache hit** (finds match): **cache** reads 1234
 - b. If **cache miss** (no match): **cache** sends address 0x12F0 to **Memory**
 - i. **Memory** reads line with 1234 @ address 0x12F0
 - ii. **Memory** sends line with 1234 to **cache**
 - iii. **Cache** replaces some line to store 1234, address 0x12F0
3. **Cache** sends 1234 to Processor
4. Processor loads 1234 into register t0





What do we need to store in the cache?

Load word instruction:

t0

1234

Memory[0x12F0] = 1234

lw t0 0(t1)

t1

0x12F0

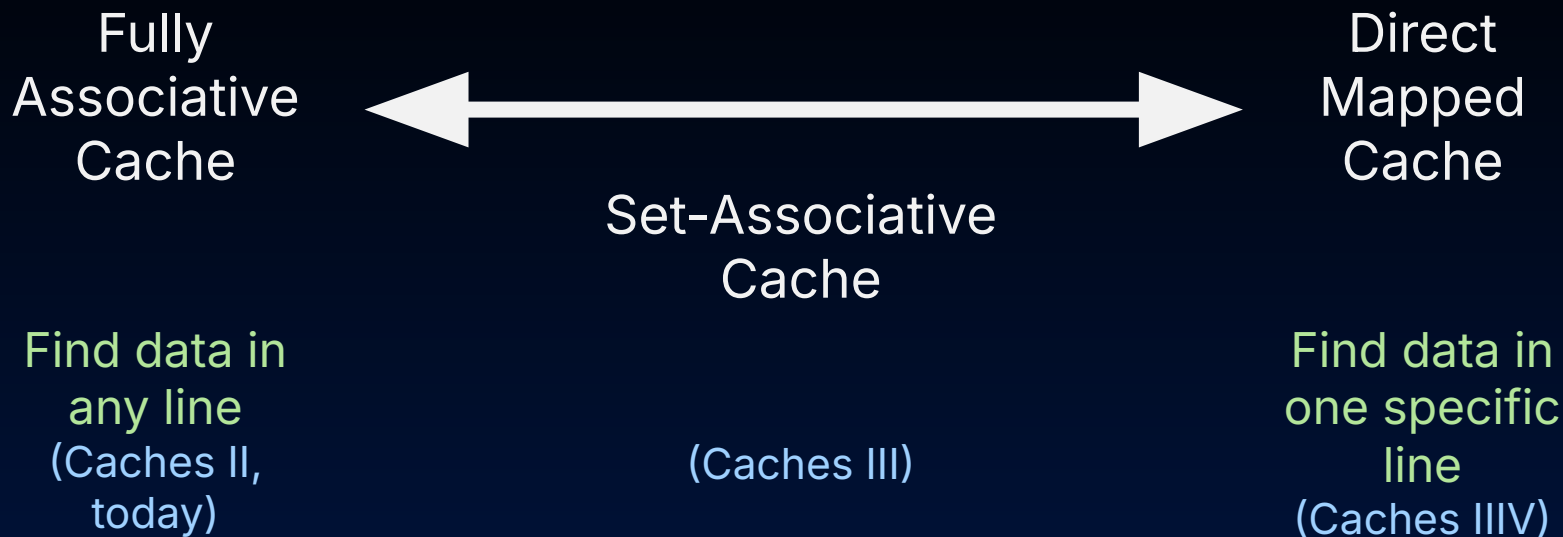
Memory access **with cache**:

1. Processor issues address 0x12F0 to **cache**
2. **Cache** checks for copy of data, addr. 0x12F0
 - a. If **cache hit** (finds match): **cache** reads 1234
 - b. If **cache miss** (no match): **cache** sends address 0x12F0 to **Memory**
 - i. **Memory** reads line with 12
 - ii. **Memory** sends line with 12
 - iii. **Cache** replaces some line
3. **Cache** sends 1234 to Processor
4. Processor loads 1234 into register t0

Cache **placement policy** determines how to associate a memory address with data in a particular cache line.



Cache Design: Placement Policies





Fully Associative Cache

- AMAT
- Cache Terminology
- Fully Associative Cache
- Warming up the Fully Associative Cache
- Line Replacement Policies
- [if time] Write Policies



Fully Associative Cache (1/2)

- Fully Associative **placement policy**:
The data can be associated with any line of the cache.
- To check if a **cache line** has data at our address:
 - Build tag and offset by splitting memory address into **tag** and **offset**
 - Check if tag matches this line's tag
- Then, access the byte by offset

Tag	Data			
	11	10	01	00
0x10F	...	...	...	...
0x178	...	...	...	...
0x209	...	...	...	...
0x149	...	...	...	...

Load byte @ 0x43F



Fully Associative Cache (2/2)

- Fully Associative **placement policy**:
The data can be associated with any line of the cache.
- To check if a **cache line** has data at our address:
 - Build tag and offset by splitting memory address into **tag** and **offset**
 - Check if tag matches this line's tag
- Then, access the byte by offset

Tag	Data			
	11	10	01	00
0x10F	...	...	...	...
0x178	...	...	...	...
0x209	...	...	...	...
0x149	...	...	...	...

Load byte @ 0x43F

Memory
address

11	2	1	0
0b0100	0011 11	1	1
0x10F		0x3	
tag		offset	



Fill in the blank: Fully Associative, 12b addresses

Suppose we have the below fully associative cache for 12b addresses.

	A.	B.	C.	D.	E.	F.
1. Line size (in bytes)	2	4	8	10	16	Other
2. Data capacity (in bytes)	2	4	8	10	16	Other
3. Bits of offset in address	2	4	8	10	16	Other
4. Bits of tag in address	2	4	8	10	16	Other

Valid	Tag	Data			
		11	10	01	00

Terminology for Fully Associative, 12b addresses

1. Line size / block size:

- # bytes per cache line

4 bytes

2. Data Capacity

- Total bytes of data that can be stored in a cache

$4 \times 4 \text{ bytes}$
= **16 bytes**

3. Offset

- Identifies byte address of data stored at a given cache line

$\log_2(\text{line size}) = \textbf{2 bits}$

4. Tag

- Identifies data stored at a given cache line

address bits - # offset bits = **10 bits**

Tag	Data			
	11	10	01	00

Memory address	11	2	1	0

tag

offset
(i.e., byte
offset)



Warming up a Fully Associative Cache

- AMAT
- Cache Terminology
- Fully Associative Cache
- Warming up the Fully Associative Cache
- Line Replacement Policies
- [if time] Write Policies



Cache Temperatures

When starting up a program, the cache necessarily does not have valid information for the program.

- Cold
 - Cache empty
- Warming
 - Cache filling with values you'll hopefully be accessing again soon
- Warm
 - Cache is doing its job, fair % of hits
- Hot
 - Cache is doing very well, high % of hits

Our goal for cache design is **temporal** and **spatial locality** for a range of workloads!



More Metadata: Valid Bit

- To indicate if the data in the cache is for this program...
- We need an indicator (i.e., **flag**) to tell if each entry is valid for this particular program.
 - **Valid bit:** Indicates if data stored at a cache line is valid
 - 1 = valid, 0 = invalid

Val id	Tag	Data			
		11	10	01	00
0	...	...	...	...	...
0	...	...	...	...	...
0	...	...	...	...	...
0	...	...	...	...	...



Warming up the Fully Associative Cache (1/6)

- Suppose the cache starts **cold**.
- 1. Load byte `0x43F` `0x10F`, `0x3`

$\overbrace{0b0100\ 0011\ 1111}^{\text{tag}\quad\text{offset}}$

Val id	Tag	Data			
		11	10	01	00
0	...	...	...	...	...
0	...	...	...	...	...
0	...	...	...	...	...
0	...	...	...	...	...

- a. **Cache miss!** Tag `0x10F` is not valid.



Warming up the Fully Associative Cache (1/6)

- Suppose the cache starts **cold**.
- Load byte 0x43F 0x10F, 0x3

0b0100 0011 1111
tag offset

Val id	Tag	Data			
		11	10	01	00
1	0x10F	...	...	...	...
0	...	...	...	...	...
0	...	...	...	...	...
0	...	...	...	...	...

- Cache miss!** Tag 0x10F is not valid.
- Load into cache the **4-byte line** from 0x43C to 0x43F.
Mark **valid** bit.

0b0100 0011 1100 ← **0x43C**

0b0100 0011 1111 ← 0x43F

Spatial locality: Even if we only read in one byte, loading from memory will load a full line's worth of data.



Warming up the Fully Associative Cache (2/6)

- Suppose the cache starts **cold**.
- 1. Load byte `0x43F` `0x10F, 0x3`

$\overbrace{0b0100\ 0011\ 1111}^{\text{tag}\quad\text{offset}}$

Val id	Tag	Data			
		11	10	01	00
1	0x10F	...	...	...	...
0	...	...	...	...	...
0	...	...	...	...	...
0	...	...	...	...	...

- a. **Cache miss!** Tag `0x10F` is not valid.
- b. Load into cache the **4-byte line** from **`0x43C` to `0x43F`**.
Mark **valid bit**.
- c. Read byte at **`0x3` offset** and return to processor.

`0b0100 0011 1100` ← **`0x43C`**

`0b0100 0011 1111` ← **`0x43E`**

Spatial locality: Even if we only read in one byte, loading from memory will load a full line's worth of data.



Warming up the Fully Associative Cache (2/6)

- Suppose the cache starts **cold**.
- 1. Load byte 0x43F 0x10F, 0x3
- 2. Load byte 0x5E2 0x178, 0x2

0b0101 1110 0010
tag offset

- a. **Cache miss!** Tag 0x178 is not valid.
- b. Load into cache the **4-byte line** from 0x5E0 to 0x5E3.
Mark **valid bit**.
- c. Read byte at **0x2 offset** and return to processor.

Val id	Tag	Data			
		11	10	01	00
1	0x10F	...	...	...	...
1	0x178	...	...	...	...
0	...	...	...	...	...
0	...	...	...	...	...

0b0101 1110 0000 ← 0x5E0
0b0101 1110 0011 ← 0x5E3

Warming up the Fully Associative Cache (2/6)

- Suppose the cache starts **cold**.
- 1. Load byte 0x43F 0x10F, 0x3
- 2. Load byte 0x5E2 0x178, 0x2
- 3. Load word 0x824 0x209, 0x0



Val id	Tag	Data			
		11	10	01	00
1	0x10F	...	...	...	...
1	0x178	...	...	...	...
1	0x209	...	...	...	...
0	...	...	...	...	...

- Cache miss!** Tag 0x209 is not valid.
- Load into cache the **4-byte line** from 0x824 to 0x827.
Mark **valid bit**.
- Read **word** at **0x0 offset** and return to processor.



Warming up the Fully Associative Cache (2/6)

- Suppose the cache starts **cold**.

- Load byte 0x43F 0x10F, 0x3
- Load byte 0x5E2 0x178, 0x2
- Load word 0x824 0x209, 0x0
- Load byte 0x5E0 0x178, 0x0

0b0101 1110 0000
tag offset

- Cache hit!** Tag 0x178 is valid.
- Read byte at **0x0 offset** and return to processor.

Val id	Tag	Data			
		11	10	01	00
1	0x10F	...	...	...	...
1	0x178	...	...	...	...
1	0x209	...	...	...	...
0	...	...	...	...	...

On a cache hit, no memory access occurs!



Line Replacement Policies

- AMAT
- Cache Terminology
- Fully Associative Cache
- Warming up the Fully Associative Cache
- Line Replacement Policies
- [if time] Write Policies



A Warmed up Cache Can Still Miss

- By the end of instr 5, cache is **warm**.
1. Load byte 0x43F 0x10F, 0x3
 2. Load byte 0x5E2 0x178, 0x2
 3. Load word 0x824 0x209, 0x0
 4. Load byte 0x5E0 0x178, 0x0
 5. Load word 0x524 0x149, 0x0

Val id	Tag	Data			
		11	10	01	00
1	0x25C	...	...	...	...
1	0x178	...	...	...	...
1	0x209	...	...	...	...
1	0x149	...	...	...	...

↖ Fills up the cache



A Warmed up Cache Can Still Miss

- By the end of instr 5, cache is **warm**.
1. Load byte 0x43F 0x10F, 0x3
 2. Load byte 0x5E2 0x178, 0x2
 3. Load word 0x824 0x209, 0x0
 4. Load byte 0x5E0 0x178, 0x0
 5. Load word 0x524 0x149, 0x0
 6. Load byte 0x972 0x25C, 0x2

0b1001 0111 0010

Cache miss!

Val id	Tag	Data			
		11	10	01	00
1	0x25C	...	...	...	...
1	0x178	...	...	...	...
1	0x209	...	...	...	...
1	0x149	...	...	...	...

When the FA cache reaches capacity, we can still miss. We need a policy for **line replacement!**



Line Replacement Policies

- Least Recently Used (LRU)
 - Replace the entry that has not been used for the longest time, i.e., has the oldest previous access.
 - Pro: **Temporal locality!**
 - recent past use implies likely future use
 - Con: Complicated hardware to keep track of access history

Fully Associative Cache with LRU policy

- By the end of instr 5, cache is **warm**.
- Load byte 0x43F 0x10F, 0x3
 - Load byte 0x5E2 0x178, 0x2
 - Load word 0x824 0x209, 0x0
 - Load word 0x5E0 0x178, 0x0
 - Load word 0x524 0x149, 0x0
 - Load byte 0x972 0x25C, 0x2

Val id	LRU	Tag	Data			
			11	10	01	00
1		0x10F	...	...	...	...
1		0x178	...	...	...	...
1		0x209	...	...	...	...
1		0x149	...	...	...	...

Suppose that LRU = 0 means **most** recently used, and 3 means **least** recently used.

After the end of instruction 5, what are the LRU tags on each row?

A.	LRU	B.	LRU	C.	LRU	D.	LRU
	0		3		3		0
	1		2		1		2
	2		1		2		3
	3		0		0		1

L16: After the end of instruction 5, what are the LRU tags on each row?

- By the end of instr 5, cache is **warm**.

- Load byte 0x43F 0x10F, 0x3
- Load byte 0x5E2 0x178, 0x2
- Load word 0x824 0x209, 0x0
- Load word 0x5E0 0x178, 0x0
- Load word 0x524 0x149, 0x0
- Load word 0x972 0x25C, 0x2

Val id	LRU	Tag	Data			
			11	10	01	00
1		0x10F	...	...	...	...
1		0x178	...	...	...	...
1		0x209	...	...	...	...
1		0x149	...	...	...	...

Suppose that LRU = 0 means **most** recently used, and 3 means **least** recently used.

After the end of instruction 5, what are the LRU tags on each row?

LRU	LRU	LRU	LRU
A. 0	B. 3	C. 3	D. 0
1	2	1	2
2	1	2	3
3	0	0	1

(A) A: 0, 1, 2, 3

0

(B) B: 3, 2, 1, 0

0

(C) C: 3, 1, 2, 0

0

(D) D: 0, 2, 3, 1

0



Fully Associative Cache with LRU policy

- By the end of instr 5, cache is **warm**.
- Load byte 0x43F 0x10F, 0x3
 - Load byte 0x5E2 0x178, 0x2
 - Load word 0x824 0x209, 0x0
 - Load word 0x5E0 0x178, 0x0
 - Load word 0x524 0x149, 0x0
 - Load byte 0x972 0x25C, 0x2

Val id	LRU	Tag	Data			
			11	10	01	00
1	3	0x10F	...	...	...	...
1	1	0x178	...	...	...	...
1	2	0x209	...	...	...	...
1	0	0x149	...	...	...	...

Suppose that LRU = 0 means **most** recently used, and 3 means **least** recently used.

After the end of instruction 5, what are the LRU tags on each row?



A Warmed up Cache Can Still Miss

- By the end of instr 5, cache is **warm**.
- 1. Load byte 0x43F 0x10F, 0x3
- 2. Load byte 0x5E2 0x178, 0x2
- 3. Load word 0x824 0x209, 0x0
- 4. Load word 0x5E0 0x178, 0x0
- 5. Load word 0x524 0x149, 0x0
- 6. Load byte 0x972 0x25C, 0x2

Val id	LRU	Tag	Data			
			11	10	01	00
1	0	0x25C	...	...	...	...
1	2	0x178	...	...	...	...
1	3	0x209	...	...	...	...
1	1	0x149	...	...	...	...

- a. **Cache miss!**
- b. Load into cache the 4-byte line from 0x970 to 0x973. Mark valid bit. Update line replacement policy fields (e.g., LRU).
- c. Read byte at 0x2 offset, return to processor.

Line Replacement Policies

- Least Recently Used (LRU)
 - Replace the entry that has not been used for the longest time, i.e., has the oldest previous access.
 - Pro: **Temporal locality**!
 - recent past use implies likely future use
 - Con: Complicated hardware to keep track of access history
- In practice: First In, First Out (FIFO)
 - Replace the oldest line in the set ("queue").
- Other policies (see in notes):
 - Most Recently Used (MRU)
 - Random
 - Last In, First Out (LIFO) - "stack"

LRU is ideal for **temporal locality** but in practice, FIFO is good enough.